

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO NHÓM 1

MÔN HỌC: SEMINAR

ĐỀ TÀI: ỨNG DỤNG MÔ HÌNH GIẢI THÍCH VIPER

Giảng viên hướng dẫn: T.S Nguyễn Quốc Huy

Sinh viên thực hiện:

Lê Minh Nguyệt	3121410355
Nguyễn Thị Xinh	3121410587
Nguyễn Văn Long	3121410299

Thành phố Hồ Chí Minh, tháng 3 năm 2025

Lời mở đầu

Trong thời đại công nghệ số ngày nay, trí tuệ nhân tạo (AI) đang dần trở thành yếu tố then chốt trong việc phát triển các hệ thống tự động hóa và ra quyết định tối ưu. Một trong những nhánh quan trọng của AI chính là Học Tăng Cường (Reinforcement Learning - RL), nơi mà các tác nhân (agents) học hỏi và cải thiện hành động của mình thông qua phản hồi từ môi trường. Tuy nhiên, mặc dù RL đã chứng minh được tiềm năng to lớn trong việc tối ưu hóa các quy trình và hệ thống phức tạp, vấn đề minh bạch và khả năng giải thích quyết định của mô hình lại là một thách thức lớn. Chính vì vậy, nghiên cứu về Học Tăng Cường Giải Thích (Explainable Reinforcement Learning - XRL) ra đời như một hướng đi quan trọng, nhằm không chỉ cải thiện hiệu suất của các mô hình RL mà còn nâng cao khả năng hiểu rõ, kiểm soát và tin tưởng vào các quyết định của hệ thống AI. Trong bối cảnh này, việc hiểu và giải thích được các quyết định của mô hình RL không chỉ là một yếu tố kỹ thuật mà còn là một yêu cầu thiết yếu trong việc ứng dụng AI vào các lĩnh vực nhạy cảm như robot tự hành, y tế, tài chính và nhiều ứng dụng quan trọng khác.

Lời cảm ơn

Đề tài này không chỉ mang đến cho chúng em những kiến thức sâu rộng về Học Tăng Cường, mà còn mở ra những hiểu biết mới mẻ và thú vị về cách thức áp dụng những lý thuyết này vào thực tiễn. Chúng em xin chân thành cảm ơn giảng viên hướng dẫn, Thầy Nguyễn Quốc Huy, người đã tận tình chia sẻ những kiến thức quý báu và giúp chúng em nhìn nhận vấn đề một cách toàn diện. Những tài liệu tham khảo từ Thầy và các chuyên gia đi trước đã là nền tảng vững chắc, giúp chúng em hoàn thiện và phát triển đề tài nghiên cứu này. Nếu không có sự sẵn lòng và hướng dẫn của Thầy, nghiên cứu này có thể sẽ không hoàn thành một cách tốt nhất.

Nhóm chúng em cũng xin gửi lời cảm ơn đến Trường đại học Sài Gòn, là nơi nền tảng để bọn em có thể thực hiện những nghiên cứu, nơi đây đã cung cấp cho nhóm em và toàn thể những học sinh về cơ sở hạ tầng, thiết bị, nơi học tập, để bọn em có thể thuận lợi hoàn thành đề tài nghiên cứu một cách tốt nhất.

Trong quá trình nghiên cứu đề tài, nhóm em không tránh khỏi những thiếu sót và sai lệch thông tin. Chúng em mong Thầy có thể thông cảm và tiếp tục chỉ dẫn để giúp chúng em khắc phục những hạn chế này. Sự đóng góp của Thầy sẽ là động lực quan trọng giúp nhóm em hoàn thiện đề tài nghiên cứu một cách tốt nhất. Một lần nữa, chúng em xin chân thành cảm ơn Thầy đã luôn đồng hành và hỗ trợ trong suốt quá trình thực hiện đề tài.

This image shows a full page of blank white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page, providing a guide for writing. There are no margins, text, or other markings on the paper.

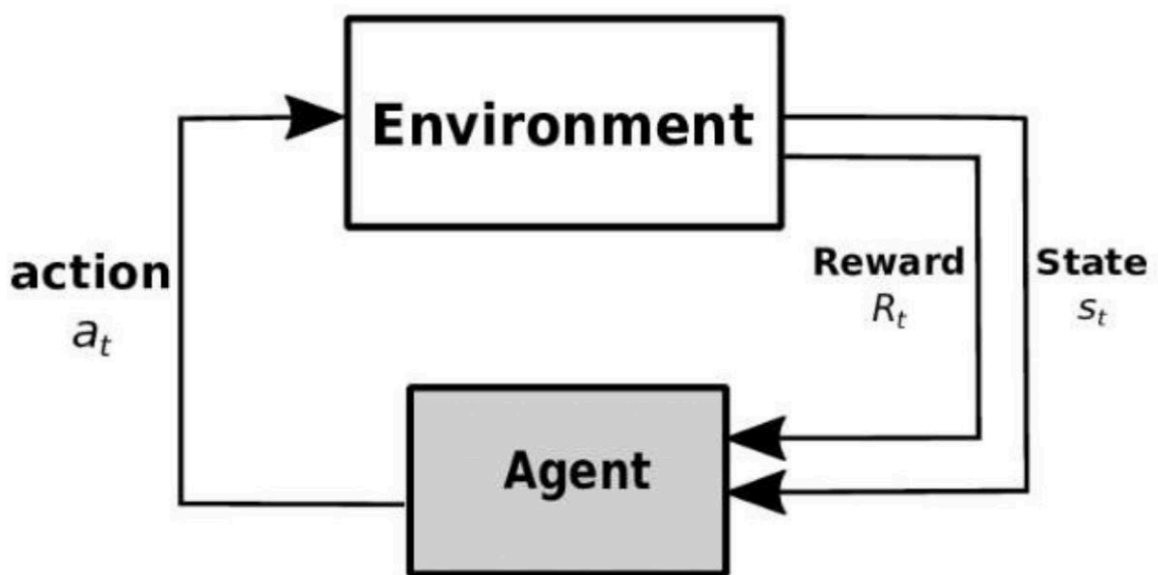
Mục lục

Lời mở đầu	2
Lời cảm ơn	3
Nhận xét của giảng viên	4
Mục lục	5
CHƯƠNG 1: Khái niệm và nền tảng về RL, XRL	7
1.1. Reinforcement Learning (RL)	7
1.2. Explainable Reinforcement Learning (XRL)	9
Chương 2: Đánh giá và giải thích hệ thống RL	11
2.1. Học tăng cường hoạt động như thế nào?	11
2.2. Ví dụ cụ thể để hiểu hơn về cách hoạt động của RL	11
Chương 3: Hệ Thống Phân Loại Mới	15
3.1. Phân loại theo khả năng diễn giải (Interpretable RL vs. Black-box RL)	15
3.2. Phân loại theo khả năng tổng quát hóa (Generalization in RL)	15
3.3. Phân loại theo mức độ tự động hóa (Auto RL)	15
3.4. Phân loại theo số lượng tác nhân (Single-Agent RL vs. Multi-Agent RL)	16
Chương 4: Tìm Hiểu Về 3 Kỹ Thuật	17
4.1. Feature Importance (FI)	17
4.1.1. Tree-based Models (Mô Hình Dựa trên Cây Quyết Định)	17
4.1.2. Permutation Feature Importance	17
4.1.3. SHAP (Shapley Additive Explanations)	18
4.1.4. L1 Regularization (Lasso Regression)	18
4.2. Local Perturbation Methods (LPM) - Phương pháp làm rối loạn cục	18
4.3. Policy Learning (PL)	23
Chương 5: Ứng Dụng Thực Tiễn	24
5.1. Ứng dụng thực tiễn vào giải mã mê cung	24
5.1.1. Bài toán	24
5.1.2. Sử dụng thuật toán Q learning	25
5.1.3. Công thức cập nhật Q table trong thuật toán Q learning:	25
5.1.4. Các bước xây dựng mô hình	26
5.1.5. Xây dựng mô hình Qlearning bằng python	26
5.1.6. Xây dựng mô hình giải thích Viper	26
5.1.7. Kết luận	31
Chương 6: Kết Luận	32

CHƯƠNG 1: Khái niệm và nền tảng về RL, XRL

1.1. Reinforcement Learning (RL)

Reinforcement Learning (RL), hay Học Tăng Cường, là một lĩnh vực trong trí tuệ nhân tạo (AI) mà trong đó, một "tác nhân" (agent) học cách hành động trong một môi trường để tối đa hóa "phần thưởng" (reward). Tác nhân này học thông qua việc tương tác với môi trường, nhận phản hồi (phần thưởng hoặc hình phạt), và từ đó cải thiện hành động của mình để đạt được mục tiêu.



Bằng cách tận dụng sức mạnh của tìm kiếm và nhiều thử nghiệm, Reinforcement Learning hiện là cách hiệu quả nhất để gợi ý sự sáng tạo của máy móc. Trái ngược với con người, trí thông minh nhân tạo có thể thu thập kinh nghiệm từ hàng nghìn gameplays song song nếu một thuật toán Reinforcement Learning được chạy trên cơ sở hạ tầng máy tính đủ mạnh.

Ví dụ: Robot dọn dẹp tự động (Robot Vacuum Cleaner)

Giả sử chúng ta có một robot dọn dẹp tự động (ví dụ, robot hút bụi). Robot này hoạt động dựa trên một mô hình Học Tăng Cường, nơi nó học cách di chuyển trong một căn phòng để dọn dẹp mọi góc ngách mà không đâm vào đồ vật hoặc các chướng ngại vật.

- + Mô hình RL: Ban đầu, robot có thể di chuyển ngẫu nhiên trong căn phòng và nhận phản hồi từ môi trường (phần thưởng nếu nó dọn dẹp thành công, hình phạt nếu nó đâm vào đồ vật).
- + Vấn đề với mô hình truyền thống: Robot sẽ học được cách di chuyển tối ưu sau một thời gian, nhưng nếu bạn hỏi nó "Tại sao bạn lại chọn di chuyển theo hướng này?", robot sẽ không thể giải thích lý do cụ thể vì mô hình là một "hộp đen".

Cấu trúc cơ bản của Học Tăng Cường:

- + Tác nhân (Agent): Là đối tượng ra quyết định trong quá trình học. Ví dụ: robot, máy tính, hay bất kỳ hệ thống nào thực hiện hành động.
- + Môi trường (Environment): Là nơi tác nhân hoạt động và nhận phản hồi. Ví dụ: một trò chơi, thị trường tài chính, hoặc một hệ thống tự động.
- + Trạng thái (State): Là tình huống hiện tại của môi trường mà tác nhân có thể quan sát được.
- + Hành động (Action): Là hành động mà tác nhân thực hiện để thay đổi trạng thái của môi trường.
- + Phần thưởng (Reward): Là phản hồi mà tác nhân nhận được sau khi thực hiện hành động. Phần thưởng có thể là số điểm, lợi nhuận, hoặc bất kỳ thứ gì có thể đo lường kết quả hành động.
- + Chính sách (Policy): Là chiến lược mà tác nhân sử dụng để chọn hành động dựa trên trạng thái hiện tại.

- Các loại học tăng cường:

- + **Củng cố tích cực:**

Học tập củng cố tích cực có nghĩa là thêm một cái gì đó để tăng xu hướng hành vi mong đợi sẽ xảy ra một lần nữa. Nó tác động tích cực đến hành vi của tác nhân và làm tăng sức mạnh của hành vi.

Loại củng cố này có thể duy trì các thay đổi trong một thời gian dài, nhưng quá nhiều củng cố tích cực có thể dẫn đến tình trạng quá tải các trạng thái có thể làm giảm hiệu quả.

+ Củng cố tiêu cực:

Việc học củng cố tiêu cực ngược lại với củng cố tích cực vì nó làm tăng xu hướng hành vi cụ thể sẽ xảy ra lần nữa bằng cách tránh điều kiện tiêu cực.

Nó có thể hiệu quả hơn sự củng cố tích cực tùy thuộc vào tình huống và hành vi, nhưng nó chỉ cung cấp sự củng cố để đáp ứng hành vi tối thiểu.

1.2. Explainable Reinforcement Learning (XRL)

Explainable Reinforcement Learning (XRL), hay Học Tăng Cường Giải Thích, là một lĩnh vực con trong Reinforcement Learning (RL), mục tiêu của XRL là làm cho các quyết định và hành động của mô hình học tăng cường trở nên dễ hiểu và giải thích được đối với con người. Trong khi các mô hình RL truyền thống có thể đưa ra các quyết định tối ưu dựa trên quá trình học hỏi từ phản hồi môi trường, chúng thường là những "hộp đen" (black box) với ít hoặc không có sự minh bạch về lý do tại sao mô hình lại đưa ra quyết định đó. XRL giúp giải thích và làm rõ những quyết định này, từ đó nâng cao tính tin cậy và khả năng kiểm soát của hệ thống.

Ví dụ: Robot dọn dẹp tự động (Robot Vacuum Cleaner) giống như ví dụ của RL

XRL trong trường hợp này sẽ giúp:

- + Giải thích hành động: Giải thích rằng robot chọn di chuyển theo hướng này vì đây là con đường ngắn nhất tới khu vực chưa được dọn dẹp và tránh được các chướng ngại vật.
- + Minh bạch trong học hỏi: XRL có thể chỉ ra rằng robot đã học cách nhận diện các khu vực bị ô nhiễm và đã nhận ra rằng hướng này sẽ giúp nó tiết kiệm thời gian dọn dẹp.

- + Giải thích chính sách: Người dùng có thể thấy rằng chính sách của robot là "luôn di chuyển về phía khu vực có bụi bẩn nhất trong phạm vi gần nhất mà không gặp phải chướng ngại vật".

- Các yếu tố quan trọng trong XRL:

- + Giải thích hành động: Giúp người dùng hiểu lý do tại sao một tác nhân trong môi trường RL lại chọn một hành động cụ thể trong một trạng thái nhất định.
- + Minh bạch trong quá trình học: Cung cấp cái nhìn rõ ràng về cách mà mô hình học hỏi và tối ưu hóa các quyết định qua thời gian.
- + Giải thích chính sách: Chỉ ra mối liên hệ giữa các trạng thái và hành động trong chính sách học của tác nhân, giúp người sử dụng dễ dàng hiểu được chiến lược mà tác nhân đang áp dụng.

Chương 2: Đánh giá và giải thích hệ thống RL

2.1. Học tăng cường hoạt động như thế nào?

Cách hoạt động của RL:

Khám phá (Exploration): là khi tác nhân thực hiện các hành động ngẫu nhiên hoặc ít thông qua các hành động mà nó chưa chắc chắn về kết quả, với mục đích tìm hiểu môi trường và tìm kiếm các chiến lược tốt hơn trong tương lai. Mục tiêu của khám phá là giúp tác nhân hiểu rõ hơn về các yếu tố ảnh hưởng đến kết quả và tìm ra những hành động có thể dẫn đến phần thưởng cao trong tương lai.

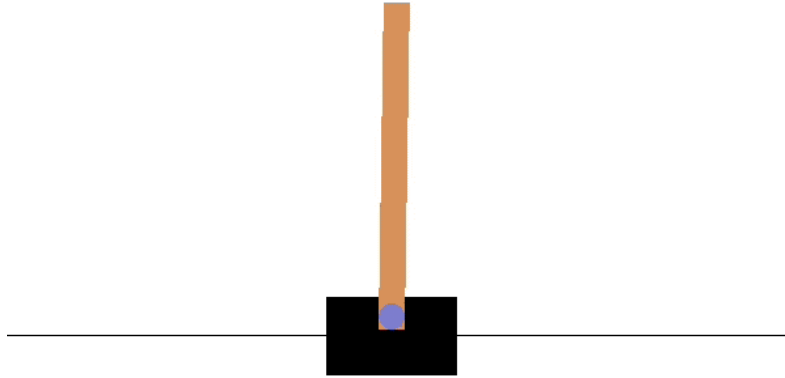
Ví dụ: Nếu một robot đang học cách di chuyển trong một phòng để tránh chướng ngại vật, robot có thể di chuyển theo những hướng mà nó chưa thử qua trước đó, chẳng hạn như đi về phía góc phòng mà nó chưa biết có vật cản hay không. Mặc dù hành động này có thể dẫn đến rủi ro (va phải vật cản), nhưng nó giúp robot tìm hiểu các phần chưa được khám phá trong môi trường. **Phần thưởng:** Nếu robot gặp vật cản, phần thưởng có thể là -1. Nếu robot tránh được vật cản, phần thưởng có thể là 0.

Khai thác (Exploitation): là khi tác nhân sử dụng thông tin và kiến thức đã học được từ những lần thử nghiệm trước để chọn hành động mà nó tin là tối ưu nhất, nhằm tối đa hóa phần thưởng trong ngắn hạn. Trong giai đoạn khai thác, tác nhân chọn các hành động mà nó cho là sẽ mang lại phần thưởng cao nhất dựa trên các trải nghiệm trước đó. Điều này có nghĩa là tác nhân sẽ tập trung vào những chiến lược đã chứng minh là hiệu quả thay vì thử nghiệm các hành động mới.

Ví dụ: Trong cùng một ví dụ với robot, sau một thời gian học hỏi, robot đã biết rằng di chuyển theo một hướng cụ thể (chẳng hạn như đi vòng quanh phòng theo chiều kim đồng hồ) sẽ giúp tránh hầu hết các vật cản. Robot sẽ tiếp tục làm theo chiến lược này vì nó đã biết đó là hành động mang lại phần thưởng cao (tránh được vật cản và dọn dẹp hiệu quả hơn).

2.2. Ví dụ cụ thể để hiểu hơn về cách hoạt động của RL

Game CartPole: Trong trò chơi này, nhiệm vụ của bạn rất đơn giản là di chuyển xe đẩy sang trái hoặc phải để giữ cây cột thăng bằng. Game kết thúc khi cây cột nghiêng quá 15 độ hoặc xe đẩy đi xa tâm quá 2.4 đơn vị. Agent giữ được cây cột càng lâu càng tốt nhưng chỉ cần đạt trung bình 195 điểm trong 100 episodes là coi như "chiến thắng".



Bước 1: Cài đặt các thư viện liên quan

Mình sẽ sử dụng keras-rl cho các module chính vì api của nó rất trực quan. Tiếp đến là gym, cung cấp cho ta Environment của rất nhiều bài toán khác nhau, trong đó có CartPole. Việc cài đặt rất đơn giản như sau:

```
pip install h5py
pip install gym
pip install keras-rl
```

Bước 2: Implement thuật toán

Import module liên quan:

```
import numpy as np
import gym
import keras
from keras.models import Sequential
from keras.layers import Dense, Activation, Flatten
from keras.optimizers import Adam

from rl.agents.dqn import DQNAgent
from rl.policy import EpsGreedyQPolicy
from rl.memory import SequentialMemory
```

Khởi tạo environment:

```
# tạo môi trường CartPole
ENV_NAME = 'CartPole-v0'
env = gym.make(ENV_NAME)
np.random.seed(123)
env.seed(123)

# số lượng hành động
nb_actions = env.action_space.n # hành động: Trái (0), Phải (1)
```

Xây dựng một Neural Network:

```
# Xây dựng mô hình mạng nơ-ron
model = Sequential()
model.add(Flatten(input_shape=(1,) + env.observation_space.shape)) #trạng thái đầu vào
model.add(Dense(32, activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(nb_actions, activation='linear')) # Đầu ra là Q-values
print(model.summary())
```

Vì bài toán này khá đơn giản nên NN của chúng ta không cần quá phức tạp; và mình thấy mạng trên cho kết quả tương đối tốt.

Phần chính: Agent, Policy và training

Mình chọn một policy đơn giản là ϵ -greedy. Policy này chọn ra action có Q-value lớn nhất với xác suất $(1-\epsilon)$ và chọn một action ngẫu nhiên với xác suất ϵ . Việc này đảm bảo agent có "mở rộng" (exploration) bên cạnh "khai thác" (exploitation).

Với Experience Replay mình giới hạn số replays lưu lại là 50000 và mỗi lần lấy ra 10 replays liên tiếp để đưa vào training.

```
# cấu hình chính sách và bộ nhớ
policy = EpsGreedyQPolicy() # Chính sách  $\epsilon$ -greedy
memory = SequentialMemory(limit=50000, window_length=1) # Replay Buffer

dqn = DQNAgent(model=model, nb_actions=nb_actions, memory=memory,
               nb_steps_warmup=10, target_model_update=1e-2, policy=policy)
dqn.compile(Adam(lr=1e-3), metrics=['mae'])

dqn.fit(env, nb_steps=20000, visualize=True, verbose=2)
```

Thử nghiệm

```
# Kiểm tra mô hình sau khi huấn luyện (visualize=True để xem)
his = dqn.test(env, nb_episodes=100, visualize=False).history['episode_reward']

print("Average reward over", len(his), "episodes:", np.sum(his) / len(his))
```

Chương 3: Hệ Thống Phân Loại Mới

3.1. Phân loại theo khả năng diễn giải (Interpretable RL vs. Black-box RL)

Học tăng cường có thể diễn giải (Interpretable RL)

- RL học ra chính sách dễ hiểu (ví dụ: sử dụng cây quyết định, hàm toán học đơn giản).
- Ví dụ:
 - + Symbolic Policy Learning (học chính sách dưới dạng biểu thức toán học).
 - + Evolutionary RL (sử dụng thuật toán tiến hóa để học chính sách dễ hiểu).
 - + Decision Tree RL (chính sách dưới dạng cây quyết định).

Học tăng cường hộp đen (Black-box RL)

- RL sử dụng mạng nơ-ron sâu để học chính sách nhưng khó giải thích.
- Ví dụ:
 - + Deep Q-Network (DQN).
 - + AlphaGo, OpenAI Five..

3.2. Phân loại theo khả năng tổng quát hóa (Generalization in RL)

Học tăng cường đặc thù (Task-Specific RL)

- RL chỉ học một nhiệm vụ cụ thể, không áp dụng được cho các bài toán khác.
- Ví dụ:
 - + DQN chơi game Atari nhưng không áp dụng được cho game khác.

Học tăng cường tổng quát (Generalizable RL)

- RL có thể học một lần và áp dụng cho nhiều nhiệm vụ.
- Ví dụ:
 - + Meta RL (RL2, MAML-RL): Học cách học nhanh từ nhiều môi trường khác nhau.
 - + Transfer RL: Học trên một bài toán rồi chuyển sang bài toán khác..

3.3. Phân loại theo mức độ tự động hóa (Auto RL)

RL truyền thống (Manual RL)

- Con người cần tinh chỉnh tham số để RL hoạt động hiệu quả.

AutoRL (Automated Reinforcement Learning)

- Hệ thống tự động tìm kiếm kiến trúc và siêu tham số tối ưu.
- Ví dụ:
 - + Neural Architecture Search (NAS) trong RL.
 - + Auto-PPO, Auto-DQN (tự động điều chỉnh siêu tham số).

3.4. Phân loại theo số lượng tác nhân (Single-Agent RL vs. Multi-Agent RL)

RL tác nhân đơn (Single-Agent RL)

- Một agent học cách tối ưu hóa chính sách cho riêng mình.

RL đa tác nhân (Multi-Agent RL - MARL)

- Nhiều agent tương tác và học từ nhau.

Chương 4: Tìm Hiểu Về 3 Kỹ Thuật

4.1. Feature Importance (FI)

- Feature Importance (FI) là một kỹ thuật trong học máy (machine learning) được sử dụng để xác định mức độ quan trọng của các đặc trưng (features) trong việc dự đoán hoặc phân loại kết quả của mô hình. Hiểu được sự quan trọng của các đặc trưng giúp chúng ta chọn lựa các yếu tố tốt nhất để cải thiện mô hình, giảm độ phức tạp và cải thiện độ chính xác của mô hình.

Các Phương Pháp Tính Feature Importance: Có nhiều cách để xác định Feature Importance trong các mô hình học máy. Dưới đây là một số phương pháp phổ biến:

4.1.1. Tree-based Models (Mô Hình Dựa trên Cây Quyết Định)

Các mô hình như Decision Trees, Random Forests, và Gradient Boosting Machines (GBM) sử dụng các kỹ thuật cây quyết định để tính toán mức độ quan trọng của từng đặc trưng.

- Decision Trees sử dụng thuật toán phân chia dữ liệu để quyết định xem đặc trưng nào giúp phân chia dữ liệu một cách tối ưu.
- Random Forests và Gradient Boosting (bao gồm XGBoost, LightGBM) tính toán Feature Importance bằng cách đánh giá các phân tách (split) dựa trên việc sử dụng các đặc trưng trong suốt các cây quyết định trong mô hình.

4.1.2. Permutation Feature Importance

Permutation Feature Importance đo lường mức độ quan trọng của một đặc trưng bằng cách làm xáo trộn giá trị của đặc trưng đó và đo lường sự thay đổi trong hiệu suất mô hình.

- Cách làm:
 - + Đo lường hiệu suất mô hình ban đầu (ví dụ, độ chính xác, F1 score, v.v.).
 - + Tiến hành xáo trộn các giá trị của một đặc trưng và đo lại hiệu suất mô hình.

- + Mức độ giảm sút hiệu suất cho thấy mức độ quan trọng của đặc trưng đó.

4.1.3. SHAP (Shapley Additive Explanations)

SHAP là một phương pháp tính toán sự quan trọng của đặc trưng dựa trên lý thuyết trò chơi Shapley, là một khái niệm trong lý thuyết kinh tế.

- + Cách làm: SHAP tính toán giá trị đóng góp của mỗi đặc trưng đối với dự đoán của mô hình, dựa trên tất cả các kết hợp có thể có của các đặc trưng trong mô hình.
- + Ưu điểm: SHAP giúp cung cấp giải thích cụ thể về ảnh hưởng của từng đặc trưng tới dự đoán của mô hình, đặc biệt là trong các mô hình phức tạp như mạng nơ-ron sâu hoặc ensemble methods.

4.1.4. L1 Regularization (Lasso Regression)

Kỹ thuật Lasso Regression (L1 Regularization) giúp chọn lựa các đặc trưng quan trọng bằng cách thêm một điều kiện hình phạt (penalty) vào hàm mất mát (loss function). Điều này giúp loại bỏ các đặc trưng không quan trọng, có thể làm cho các hệ số của các đặc trưng không quan trọng trở thành bằng 0.

- + Cách làm: Trong Lasso Regression, các đặc trưng có hệ số nhỏ hoặc bằng 0 sẽ bị loại bỏ, giúp giảm số lượng đặc trưng cần thiết trong mô hình.

4.2. Local Perturbation Methods (LPM) - Phương pháp làm rối loạn cục

Local Perturbation Methods (LPM) là một nhóm kỹ thuật trong lĩnh vực học máy (machine learning) và giải thích mô hình (model interpretability) giúp hiểu rõ hơn về cách mà mô hình đưa ra quyết định cho một dự đoán cụ thể. Thực chất, các phương pháp này tạo ra các biến động cục bộ trong không gian đặc trưng để kiểm tra ảnh hưởng của chúng đến dự đoán của mô hình, từ đó cung cấp thông tin về các yếu tố quyết định quan trọng trong dự đoán.

Các phương pháp Local Perturbation thường được sử dụng khi muốn giải thích hành vi của mô hình tại một điểm dữ liệu cụ thể mà không cần phải hiểu toàn bộ mô hình. Điều này rất hữu ích khi bạn làm việc với các mô hình phức tạp như mạng

neuron sâu (deep neural networks) hoặc mô hình ensemble mà không thể giải thích trực tiếp.

Cách thức hoạt động của LPM

1. Tạo ra Perturbations (Biến động):

- Phương pháp LPM thực hiện biến động cục bộ trên dữ liệu đầu vào. Cụ thể, nó sẽ thay đổi (hoặc làm rối loạn) các giá trị của các đặc trưng ở một mẫu dữ liệu cụ thể.
- Các thay đổi này thường được thực hiện một cách nhỏ (perturbations nhẹ) và ngẫu nhiên trong không gian đặc trưng.
- Mục đích là kiểm tra cảm biến (sensitivity) của mô hình đối với các thay đổi nhỏ này, qua đó hiểu được những yếu tố quan trọng ảnh hưởng đến quyết định của mô hình tại điểm dữ liệu đó.

2. Đo lường ảnh hưởng của Perturbation:

- Sau khi tạo ra các biến động, mô hình sẽ được yêu cầu đưa ra dự đoán mới đối với các điểm dữ liệu đã bị làm rối loạn.
- Sự thay đổi trong các dự đoán được quan sát giúp xác định mức độ ảnh hưởng của các đặc trưng đã bị biến động. Nếu một đặc trưng bị thay đổi và mô hình thay đổi dự đoán một cách rõ rệt, điều này cho thấy đặc trưng đó có ảnh hưởng lớn đến quyết định của mô hình.

3. Giải thích các Dự đoán:

- Sau khi thực hiện các perturbation, kết quả sẽ giúp ta hiểu rõ hơn về các yếu tố quyết định mô hình khi dự đoán cho các điểm dữ liệu cụ thể.
- Nếu một mô hình học sâu không thể dễ dàng giải thích, LPM có thể giúp chúng ta hiểu tại sao mô hình lại đưa ra một dự đoán cụ thể cho một dữ liệu đầu vào.

Các Phương Pháp Cụ Thể của LPM

1. LIME (Local Interpretable Model-agnostic Explanations):

- LIME là một trong những phương pháp phổ biến nhất thuộc nhóm Local Perturbation Methods. LIME hoạt động bằng cách làm rối loạn các đặc trưng trong dữ liệu đầu vào và xây dựng một mô hình đơn giản (thường là mô hình tuyến tính) để giải thích quyết định của mô hình phức tạp tại điểm dữ liệu đó.
- **LIME** là một phương pháp giải thích các mô hình học máy phức tạp (như deep learning, random forests, hay support vector machines) một cách dễ hiểu. Mục tiêu của LIME là tạo ra những giải thích dễ tiếp cận về cách mà các mô hình đưa ra quyết định, trong khi vẫn giữ được sự chính xác của mô hình.

- **Nguyên lý hoạt động của LIME:**

1. Mô hình đen (Black-box model): LIME được thiết kế để giải thích các mô hình học máy phức tạp mà không cần biết rõ cấu trúc của chúng. Các mô hình này được gọi là black-box models vì chúng có thể rất khó hiểu, ví dụ như deep neural networks hay random forests.

2. Giải thích cục bộ (Local Explanations): Thay vì cố gắng giải thích toàn bộ mô hình (global explanations), LIME tập trung vào việc giải thích các quyết định của mô hình đối với các mẫu dữ liệu cụ thể. Đây là giải thích cục bộ, có nghĩa là LIME chỉ quan tâm đến hành động của mô hình xung quanh điểm dữ liệu mà chúng ta muốn giải thích.

3. Quy trình hoạt động:

- LIME tạo ra các phiên bản thay thế của dữ liệu mẫu gần mẫu cần giải thích. Những phiên bản thay thế này có thể được làm mờ dần từ dữ liệu thật (bằng cách thay đổi một vài đặc trưng).
- LIME sau đó sẽ huấn luyện một mô hình đơn giản (thường là hồi quy tuyến tính hoặc cây quyết định) để dự đoán kết quả cho các phiên bản thay thế này.
- Cuối cùng, mô hình đơn giản này được sử dụng để giải thích quyết định của mô hình đen (black-box model). Bởi vì mô hình đơn giản dễ hiểu hơn, nên LIME giúp ta hiểu được lý do tại sao mô hình đen lại đưa ra quyết định cho mẫu dữ liệu đó.

- **Cách LIME hoạt động:**

Lựa chọn mẫu dữ liệu cần giải thích: Đầu tiên, chúng ta chọn một mẫu dữ liệu mà chúng ta muốn giải thích.

Tạo các phiên bản thay thế: LIME tạo ra các phiên bản thay thế gần với mẫu dữ liệu gốc. Các phiên bản này có thể có một số đặc trưng bị thay đổi hoặc làm mờ dần.

Huấn luyện mô hình đơn giản: LIME huấn luyện một mô hình đơn giản (thường là hồi quy tuyến tính) trên những phiên bản thay thế này. Mô hình này sẽ cố gắng bắt chước hành vi của mô hình phức tạp (black-box model) trong vùng xung quanh mẫu dữ liệu cần giải thích.

Giải thích quyết định của mô hình: Mô hình đơn giản được huấn luyện trên các phiên bản thay thế này sẽ chỉ ra lý do tại sao mô hình phức tạp lại đưa ra quyết định cho mẫu gốc.

- **Ví dụ về LIME:**

Giả sử bạn đang sử dụng một mô hình học sâu (neural network) để phân loại các bức ảnh thành các lớp như "mèo", "chó", "người". Bạn muốn giải thích lý do tại sao mô hình lại phân loại một bức ảnh là "mèo".

1. **Chọn mẫu cần giải thích:** Chọn bức ảnh mà mô hình phân loại là "mèo".
2. Tạo các phiên bản thay thế: LIME tạo ra nhiều phiên bản của bức ảnh này bằng cách thay đổi một số pixel ngẫu nhiên hoặc làm mờ đi các phần của bức ảnh.
3. **Huấn luyện mô hình đơn giản:** Sau đó, LIME huấn luyện một mô hình đơn giản (chẳng hạn như hồi quy tuyến tính) để dự đoán lớp (mèo, chó, người) cho những phiên bản thay thế này.
4. **Giải thích quyết định:** Mô hình đơn giản này có thể chỉ ra rằng các phần quan trọng trong bức ảnh như tai, mũi hoặc mắt là yếu tố quyết định giúp mô hình phân loại bức ảnh là "mèo".

2. SHAP (Shapley Additive Explanations):

- SHAP là một phương pháp mạnh mẽ giúp giải thích mô hình học máy, đặc biệt là các mô hình phức tạp. Mặc dù SHAP không phải là một phương pháp perturbation thuần túy, nhưng SHAP tính toán sự đóng góp của từng đặc trưng

đối với dự đoán của mô hình, giống như việc làm rối loạn các giá trị đặc trưng và đo lường sự thay đổi trong dự đoán.

Ví Dụ Cụ Thể Về LPM

Giả sử chúng ta đang sử dụng một mô hình học máy để dự đoán khả năng người dùng có trả nợ hay không dựa trên các đặc trưng như:

- + Thu nhập
- + Số năm làm việc
- + Lịch sử tín dụng
- + Tuổi

Một cách đơn giản, chúng ta có thể sử dụng LIME để giải thích mô hình học máy về dự đoán "Có trả nợ hay không" cho một cá nhân cụ thể:

1. Bước 1: Lựa chọn mẫu dữ liệu.

- Chọn một mẫu dữ liệu, ví dụ, một người có thu nhập 50,000 đô la, đã làm việc 5 năm, và có điểm tín dụng 700.

2. Bước 2: Tạo các perturbation.

- LIME tạo ra các perturbation bằng cách thay đổi một cách ngẫu nhiên giá trị của các đặc trưng như thu nhập, số năm làm việc, hoặc lịch sử tín dụng. Chẳng hạn, thay vì thu nhập 50,000 đô la, một perturbation có thể tạo ra giá trị thu nhập mới là 55,000 đô la.

3. Bước 3: Dự đoán và phân tích.

- Mô hình học máy sẽ dự đoán lại kết quả cho các dữ liệu đã bị biến động này. Nếu việc thay đổi thu nhập hoặc lịch sử tín dụng có ảnh hưởng lớn đến dự đoán, **LIME** sẽ ghi nhận điều này và xác định rằng **thu nhập** hoặc **lịch sử tín dụng** là đặc trưng quan trọng trong việc quyết định khả năng trả nợ.

4. Bước 4: Giải thích kết quả.

- Dựa vào kết quả perturbation và các dự đoán, **LIME** sẽ cung cấp một mô hình giải thích (chẳng hạn, mô hình tuyến tính) để cho thấy các đặc trưng nào có ảnh hưởng lớn nhất đến quyết định của mô hình học máy.

4.3. Policy Learning (PL)

- Policy Learning (PL) là một phương pháp quan trọng trong học tăng cường (Reinforcement Learning - RL) nhằm học được chính sách (policy) tối ưu để tác nhân (agent) có thể đưa ra các hành động nhằm tối đa hóa phần thưởng trong môi trường. Chính sách (policy) ở đây là một hàm ánh xạ từ trạng thái của môi trường (state) đến các hành động mà tác nhân sẽ thực hiện.

Các Phương Pháp Chính trong Policy Learning

1. Policy Gradient Methods (Phương pháp Gradient Chính Sách):

- Policy Gradient Methods trực tiếp tối ưu hóa chính sách bằng cách điều chỉnh các tham số của chính sách dựa trên gradient của phần thưởng kỳ vọng. Thay vì học hàm giá trị và từ đó suy ra chính sách, phương pháp này học trực tiếp chính sách từ các hành động và phần thưởng mà tác nhân nhận được.
- Một ví dụ của phương pháp này là REINFORCE, một thuật toán dùng để cập nhật chính sách dựa trên gradient của phần thưởng.
- Ưu điểm: Phương pháp này có thể hoạt động tốt đối với các không gian hành động liên tục và rất hữu ích khi không thể dễ dàng tính toán hàm giá trị.

2. Actor-Critic Methods:

- Actor-Critic kết hợp cả hai phương pháp: Actor (tác nhân) và Critic (người đánh giá). Actor học chính sách (policy) trong khi Critic học hàm giá trị (value function).
- Actor sẽ đưa ra hành động dựa trên chính sách, và Critic sẽ đánh giá hành động đó dựa trên phần thưởng thực tế và hàm giá trị. Actor sau đó điều chỉnh chính sách của mình dựa trên đánh giá của Critic.
- Ưu điểm: Phương pháp này giúp cân bằng giữa việc học chính sách và việc đánh giá chính sách, giúp cải thiện tốc độ hội tụ.

3. Q-Learning (Học Q):

- Q-Learning là một phương pháp học không cần chính sách (off-policy), nơi tác nhân học một hàm giá trị $Q(s,a)$ để dự đoán giá trị của mỗi cặp trạng thái-hành động. Dựa trên giá trị này, tác nhân sẽ lựa chọn hành động tốt nhất tại mỗi trạng thái.
- Ưu điểm: Q-Learning có thể học chính sách tối ưu mà không cần mô hình đầy đủ của môi trường và có thể hoạt động tốt trong môi trường có không gian hành động rời rạc.

4. Deep Reinforcement Learning (Học Tăng Cường Sâu):

- Trong Deep Reinforcement Learning, các mô hình học sâu (deep learning) được sử dụng để học chính sách, đặc biệt là trong các môi trường phức tạp với không gian trạng thái và hành động rất lớn.
- Một trong những kỹ thuật phổ biến là Deep Q-Network (DQN), kết hợp giữa Q-Learning và học sâu để học chính sách trong môi trường có không gian trạng thái lớn.

Chương 5: Ứng Dụng Thực Tiễn

5.1. Ứng dụng thực tiễn vào giải mã mê cung

5.1.1. Bài toán

Game mê cung là một trò chơi giải đố, trong đó người chơi cần tìm đường đi từ điểm bắt đầu (thường là góc trái trên cùng bản đồ) đến đích (điểm thoát góc dưới bên phải bản đồ) mà không đi qua tường.

Game mê cung thường có nhiều lối đi khác nhau, và trong ứng dụng này, nhóm em đang hướng tới mục tiêu tìm đường đi tối ưu nhất.

Trạng thái:

- Vị trí của agent trên bản đồ

Hành động

- Agent di chuyển lên xuống, qua trái, qua phải trên bản đồ mê cung

Phần thưởng:

- Mỗi lần di chuyển, phần thưởng sẽ -10
- Khi di chuyển sang vị trí là tường thì -1000
- Khi di chuyển sang vị trí là bom thì -100
- Khi đến đích sẽ được +500

5.1.2. Sử dụng thuật toán Q learning

Thuật toán Q-learning là một phương pháp học tăng cường (Reinforcement Learning - RL), rất phù hợp để giải quyết bài toán tìm đường trong mê cung.

Ưu điểm:

- Không giống như thuật toán lập kế hoạch như *A* hoặc *Dijkstra**, Q-learning không cần phải biết trước cấu trúc của mê cung, thay vào đó, thuật toán có thể học từ kinh nghiệm
- Q-learning giúp tìm ra chính sách tối ưu để đi từ điểm xuất phát đến đích bằng cách thử nghiệm nhiều lần và cập nhật giá trị hành động tốt nhất, nó không chỉ tìm một lời giải bất kỳ mà còn có thể tìm đường đi ngắn nhất hoặc hiệu quả nhất.

Nhược điểm:

- Nếu mê cung quá lớn, việc lưu bảng Q-table có thể không khả thi do chi phí bộ nhớ cao

Phương án giải quyết: mở rộng Q-learning bằng Deep Q-Network (DQN), sử dụng mạng nơ-ron để ước lượng giá trị Q thay vì lưu trữ bảng.

5.1.3. Công thức cập nhật Q table trong thuật toán Q learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Trong đó:

- s là trạng thái hiện tại (vị trí trong mê cung)
- a là hành động (đi lên, xuống, trái, phải)
- R là phần thưởng nhận được khi thực hiện hành động

- $\alpha = 0.3$ là hệ số học (learning rate)
- $\gamma = 0.95$ là hệ số chiết khấu (discount factor)
- s' là trạng thái tiếp theo
- a' là hành động tối ưu từ trạng thái

5.1.4. Các bước xây dựng mô hình

Các bước xây dựng mô hình Q_Learning huấn luyện agent

- **B1:** Khởi tạo Q-table là một ma trận chứa các giá trị ban đầu.
- **B2:** Lựa chọn hành động: Agent di chuyển trong mê cung dựa trên chính sách epsilon-greedy (đôi khi chọn hành động tốt nhất, đôi khi chọn ngẫu nhiên để khám phá).
- **B3:** Tổng hợp thưởng: Mỗi lần di chuyển, agent nhận thưởng (+500 khi đến đích, -1 khi di chuyển, -15 nếu đâm vào tường).
- **B4:** Cập nhật Q-table: Dựa trên các giá trị nhận được.
- **B5:** Lặp lại trong nhiều lần huấn luyện để cải thiện đường đi.

Các bước xây dựng mô hình giải thích Viper

- **B1:** Lấy dữ liệu huấn luyện cho cây quyết định từ quá trình học của agent
- **B2:** Xây dựng 1 mô hình cây quyết định
- **B3:** Lấy dữ liệu đường đi tối ưu của agent.
- **B4:** Dùng Viper giải thích cho từng lựa chọn của agent theo các luật của cây quyết định.

5.1.5. Xây dựng mô hình Qlearning bằng python

Nguồn code xây dựng mê cung:

<https://github.com/PPoonYaPat/ESTR2018-Markov-Decision-Process>

5.1.6. Xây dựng mô hình giải thích Viper

Chuẩn bị dữ liệu:

Dữ liệu đào tạo gồm 12 đặc trưng và 1 nhãn hành động được lấy từ quá trình huấn luyện của agent.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	x	y	x_g	y_g	wall_top	wall_bot	wall_left	wall_right	bomb_top	bomb_bot	bomb_left	bomb_right	near_top	near_bot	near_left	near_right	action
2	16	16	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	TRUE	TRUE	FALSE	up
3	15	15	19	13	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
4	14	12	19	13	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	up
5	15	13	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	up
6	15	13	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	up
7	15	15	19	13	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	left
8	16	12	19	13	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	up
9	15	13	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	up
10	17	11	19	13	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	up
11	15	11	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	up
12	3	17	19	13	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
13	1	17	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
14	1	18	19	13	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
15	0	19	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	TRUE	FALSE	down
16	1	17	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
17	0	18	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	down
18	3	17	19	13	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
19	7	15	19	13	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
20	9	15	19	13	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
21	10	16	19	13	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	up
22	2	4	19	13	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	up
23	3	7	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	up
24	4	4	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	up
25	4	2	19	13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	TRUE	up
26	7	3	19	13	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	up

Trong các giai đoạn đầu huấn luyện, agent thường chọn hành động ngẫu nhiên hoặc sai do chưa học được gì nhiều. Các hành động lúc đó không phản ánh chính sách tối ưu.

Khi lấy dữ liệu ở cuối quá trình huấn luyện, bạn thu thập được các trạng thái – hành động mà agent tin là tốt nhất → dữ liệu sạch hơn và gần chính sách mục tiêu, dẫn đến cây quyết định chính xác hơn.

Mô hình cây quyết định:

```
|--- wall_bot <= 0.50
|   |--- near_left <= 0.50
|   |   |--- near_bot <= 0.50
|   |   |   |--- class: right
|   |   |   |--- near_bot > 0.50
|   |   |   |--- bomb_bot <= 0.50
|   |   |   |   |--- bomb_right <= 0.50
|   |   |   |   |   |--- class: down
|   |   |   |   |   |--- bomb_right > 0.50
|   |   |   |   |   |--- class: down
|   |   |   |--- bomb_bot > 0.50
|   |   |   |   |--- bomb_right <= 0.50
|   |   |   |   |   |--- class: right
|   |   |   |   |   |--- bomb_right > 0.50
```

```

|   |   |   |   |   |--- class: down
|   |--- near_left >  0.50
|   |   |--- wall_left <= 0.50
|   |   |   |--- bomb_bot <= 0.50
|   |   |   |   |--- bomb_left <= 0.50
|   |   |   |   |   |--- class: down
|   |   |   |   |   |--- bomb_left >  0.50
|   |   |   |   |   |--- class: down
|   |   |   |--- bomb_bot >  0.50
|   |   |   |   |--- wall_right <= 0.50
|   |   |   |   |   |--- class: left
|   |   |   |   |   |--- wall_right >  0.50
|   |   |   |   |   |--- class: left
|   |   |--- wall_left >  0.50
|   |   |   |--- near_bot <= 0.50
|   |   |   |   |--- class: up
|   |   |   |   |--- near_bot >  0.50
|   |   |   |   |--- bomb_right <= 0.50
|   |   |   |   |   |--- class: down
|   |   |   |   |   |--- bomb_right >  0.50
|   |   |   |   |   |--- class: up
|--- wall_bot >  0.50
|   |--- wall_left <= 0.50
|   |   |--- near_left <= 0.50
|   |   |   |--- wall_right <= 0.50
|   |   |   |   |--- wall_top <= 0.50
|   |   |   |   |   |--- class: right
|   |   |   |   |   |--- wall_top >  0.50
|   |   |   |   |   |--- class: right
|   |   |   |--- wall_right >  0.50
|   |   |   |   |--- class: left
|   |   |--- near_left >  0.50
|   |   |   |--- bomb_left <= 0.50
|   |   |   |   |--- wall_top <= 0.50
|   |   |   |   |   |--- class: left
|   |   |   |   |   |--- wall_top >  0.50
|   |   |   |   |   |--- class: left
|   |   |   |--- bomb_left >  0.50
|   |   |   |   |--- bomb_right <= 0.50

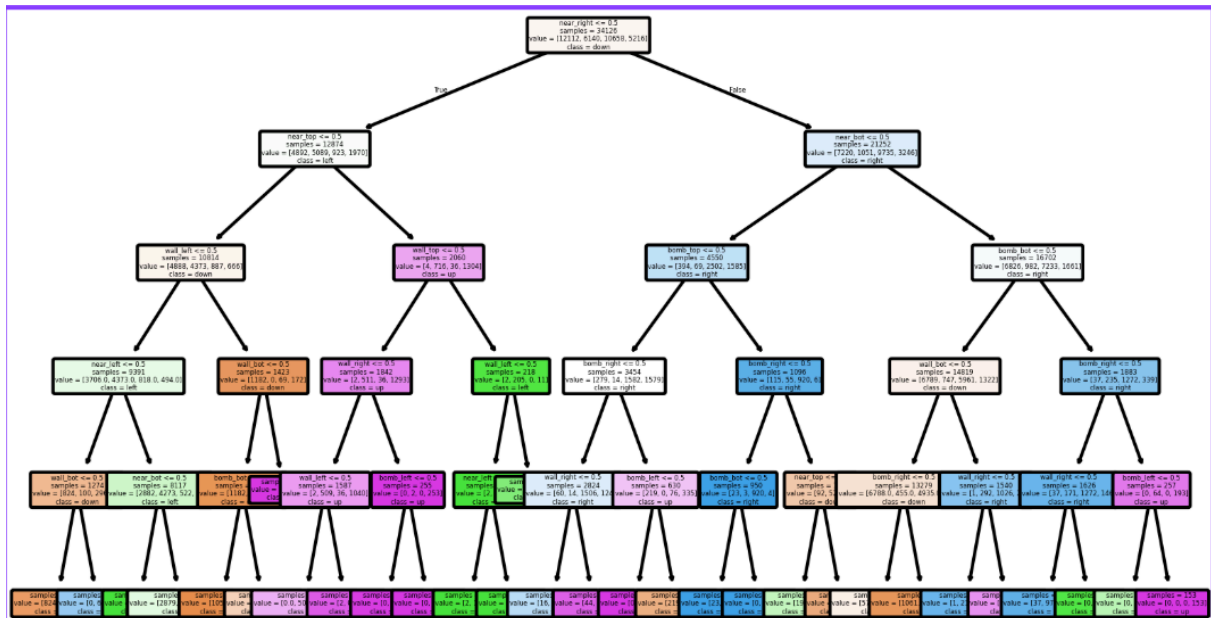
```

```

|   |   |   |   |   |   |--- class: up
|   |   |   |   |   |--- bomb_right > 0.50
|   |   |   |   |   |--- class: right
|   |--- wall_left > 0.50
|   |   |--- bomb_top <= 0.50
|   |   |   |--- wall_right <= 0.50
|   |   |   |   |--- class: up
|   |   |   |   |--- wall_right > 0.50
|   |   |   |   |--- class: up
|   |   |--- bomb_top > 0.50
|   |   |   |--- class: right

```

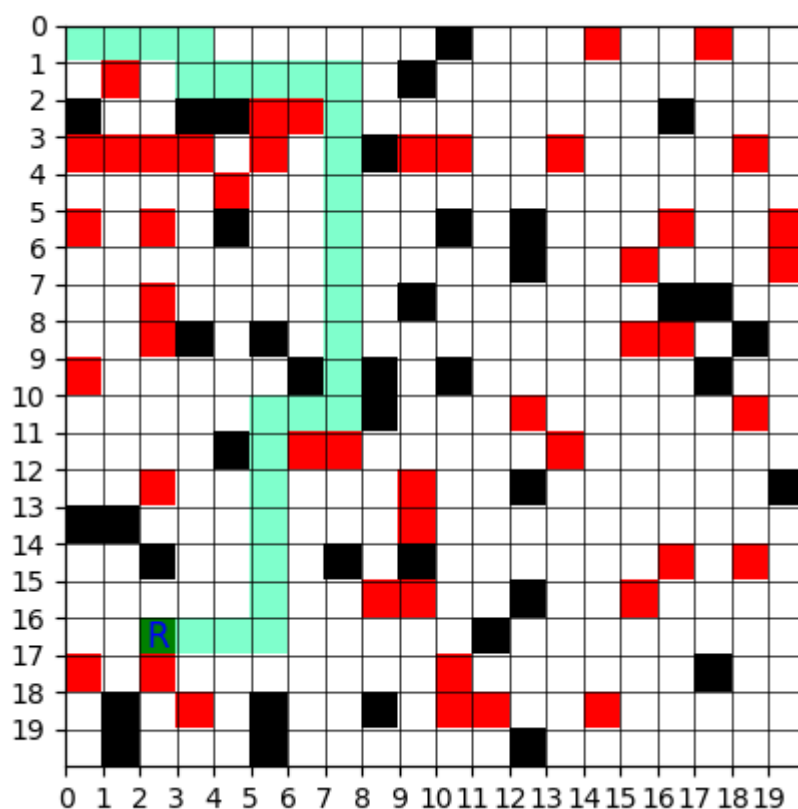
Trực quan hóa cây quyết định bằng hình ảnh:



Lấy dữ liệu đường đi tối ưu của agent để giải thích

	x	y	x_g	y_g	wall_top	wall_bot	...	bomb_right	near_top	near_bot	near_left	near_right	action
0	0	0	6	4	False	False	...	False	False	True	False	True	right
1	0	1	6	4	False	False	...	False	False	True	False	True	right
2	0	2	6	4	False	False	...	False	False	True	False	True	down
3	1	2	6	4	False	False	...	False	False	True	False	True	right
4	1	3	6	4	False	True	...	False	False	True	False	True	right
5	1	4	6	4	False	False	...	False	False	True	False	False	down
6	2	4	6	4	False	True	...	False	False	True	False	False	right
7	2	5	6	4	False	False	...	False	False	True	True	False	down
8	3	5	6	4	False	False	...	True	False	True	True	False	down
9	4	5	6	4	False	False	...	True	False	True	True	False	down
10	5	5	6	4	False	False	...	False	False	True	True	False	left
11	5	4	6	4	False	False	...	False	False	True	False	False	down

Mô hình Viper giải thích cho các tình huống nhất định



===== Trường hợp [10, 7] =====

Thực tế Agent 'left'

🌳 (Luật đầy đủ) Agent chọn left vì: agent xuống dưới gần đích hơn, phía dưới không có tường, phía dưới có bomb, agent qua trái gần đích hơn, bên trái không có tường,

🌳 Agent chọn 'left' vì: agent qua trái gần đích hơn, bên trái không có tường,

⊘ Agent không chọn 'up' vì: agent lên trên không gần đích hơn,

⊘ Agent không chọn 'down' vì: agent xuống dưới gần đích hơn, phía dưới có bomb,

⊘ Agent không chọn 'right' vì: bên phải có tường,

5.1.7. Kết luận

Trò chơi này là một ứng dụng AI thú vị, kết hợp giữa Pygame và Q-learning để giải quyết bài toán tìm đường trong mê cung.

Tuy nhiên, khi khởi tạo mê cung với kích thước quá lớn, mô hình không thực sự hoạt động hiệu quả. Cần kết hợp với các mô hình khác để đạt được lời giải mong muốn.

Về phần mô hình giải thích, vì còn thiếu kỹ năng và kiến thức, nhóm chúng em chưa xây dựng được mô hình giải thích những lý do không chọn hành động của agent. Chúng em sẽ cố gắng hoàn thiện trong thời gian sắp tới.

Chương 6: Kết Luận

Trong bài báo cáo này, chúng ta đã đi qua một hành trình tìm hiểu về Học Tăng Cường (Reinforcement Learning - RL) và Học Tăng Cường Giải Thích (Explainable Reinforcement Learning - XRL), cùng với các kỹ thuật và ứng dụng thực tế. Chương 6 tóm tắt các kết quả và đóng góp của nghiên cứu này.

Qua đó, chúng ta đã thấy được sự quan trọng của việc hiểu và giải thích các quyết định của các mô hình RL, đặc biệt trong các hệ thống tự động như robot tự hành, trò chơi và các ứng dụng trong ngành công nghiệp. Các phương pháp như Policy Learning (PL) và Local Perturbation Methods (LPM), kết hợp với kỹ thuật Feature Importance, cung cấp những công cụ mạnh mẽ để giúp chúng ta không chỉ tối ưu hóa hành vi của tác nhân mà còn giải thích được các quyết định của mô hình.

Đặc biệt, các ứng dụng thực tiễn như giải mã mê cung và MountainCar đã chứng minh tính khả thi và hiệu quả của Q-learning trong việc giải quyết các bài toán thực tế, đồng thời giúp hiểu rõ hơn về quá trình học của tác nhân. Những thử nghiệm này không chỉ nâng cao khả năng của các hệ thống tự động mà còn giúp các nhà nghiên cứu và kỹ sư cải thiện tính minh bạch của các mô hình AI.

Cuối cùng, nghiên cứu này không chỉ là một bước tiến trong việc hiểu rõ hơn về RL và XRL mà còn là nền tảng để phát triển các hệ thống AI hiệu quả và đáng tin cậy trong tương lai. Những hướng nghiên cứu tiếp theo có thể tập trung vào việc cải thiện sự chuyển giao học tập giữa các nhiệm vụ, tối ưu hóa hiệu suất học tăng cường, và tạo ra những phương pháp giải thích tốt hơn cho các mô hình AI trong những tình huống phức tạp.